

Coding & Solution

Date	02 July 2026
Team ID	
Project Name	Rising Waters – A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented code and the overall solution, demonstrating working functionality, clean code practices, and alignment with the defined requirements.

Solution Summary

- **Notebook (Flood_Prediction_Analysis.ipynb):** 46-cell pipeline covering environment setup, dataset collection, EDA on two datasets, pre-processing (median imputation, IQR outlier capping, StandardScaler), training/comparison of four classifiers, and model persistence via joblib.
- **Flask application (app/app.py, 143 lines):** loads the model bundle once at startup, exposes / and /predict routes, validates form input per feature, scales inputs, runs prediction, and renders the appropriate result template.
- **Templates:** base.html (shared layout), home.html (overview + live accuracy stats), predict.html (grouped input form), flood_result.html / no_flood_result.html (outcome pages).
- **Deployment artifacts:** Dockerfile (Code Engine), manifest.yml (Cloud Foundry), Procfile (generic buildpack PaaS) — all three ready to use.

Key Implementation Details

- **Graceful XGBoost fallback:** the notebook attempts import xgboost and automatically substitutes scikit-learn's GradientBoostingClassifier if it isn't installed, so the pipeline always runs end-to-end.
- **Automatic best-model selection:** the four trained classifiers are compared on identical train/test splits and the highest-accuracy model is chosen, preferring the boosted ensemble on ties for better generalization.
- **Single source of truth for features:** FEATURE_NAMES is read directly from the saved bundle, so the prediction form and model always stay in sync without duplicated feature lists.
- **Input validation:** each of the 10 form fields is parsed with try/except float conversion; invalid entries produce field-specific error messages instead of a server crash.

Instructions Followed

- Code is structured, commented (module docstring, inline comments) and follows idiomatic Flask/scikit-learn practices for the chosen stack.
- The solution directly addresses the core problem statement (fast, data-driven flood-risk assessment) and implements all functional requirements listed in the Solution Requirements document.

- All source files (data/, notebook/, model/, app/, deployment configs) are included in the repository as required for evaluation and reproduction.